# PakWheels Used Car Price Predictor

AI-Powered Price Prediction with Explainable AI

Updated Final Report — All Deliverables

Phases 1–6: Problem Definition, Data, Modelling, Evaluation, XAI & Deployment

**Live Deployment**

https://muhammad-ahmad2511-used-car-price-predictor.streamlit.app

Muhammad Ahmad 23L-2511

Muhammad Sufyan 23L-2518

Muhammad Daniyal 23L-2600

Course: Artificial Intelligence Track: Track A — Application Development

Submission Date: May 2026

GitHub: https://github.com/Muhammad-Ahmad2511/used-car-price-predictor

# Abstract

Pakistan's used car market suffers from significant price opacity and information asymmetry, creating adverse conditions for buyers, sellers, and financial institutions alike. This report presents a complete end-to-end AI solution — **PakWheels Used Car Price Predictor** — that addresses these challenges through machine learning-based price estimation combined with Explainable AI (XAI). We scraped 24,728 listings from PakWheels.com, cleaned and engineered 26 features, and trained two gradient boosting models: XGBoost and LightGBM. Our best model, LightGBM, achieves an $R^2$ of 0.873 and a Mean Absolute Error of Rs 562,417 on the held-out test set. SHAP values provide per-prediction transparency, revealing that model, engine_cc, and mileage_per_year are the dominant price drivers. The complete system is deployed as a public Streamlit web application with real-time inference and interactive SHAP explanations. Post-submission, the application UI was enhanced with encoder-aware model/make dropdowns, intelligent input validation, and a flexible “Other” model mode — all described in the new Section 7 (Application Updates).


# 1. Phase 1: Problem Definition

## 1.1 Problem Statement

Pakistan's used car market suffers from significant price opacity and information asymmetry, creating challenges for all stakeholders. The market is characterised by:

**Price Inconsistency:** Similar vehicles exhibit price variations up to 30% across sellers and cities.

**Information Asymmetry:** Sellers possess complete vehicle history while buyers rely on incomplete information.

**Market Fragmentation:** The market spans multiple cities with varying demand–supply dynamics.

**Limited Transparency:** Traditional valuation relies on subjective dealer assessments.

## 1.2 Target Audience

**Individual Buyers (70%):** Fair price estimation, confidence in decisions, understanding price factors.

**Car Dealers & Showrooms (20%):** Competitive pricing strategies, inventory valuation, market trend analysis.

**Financial Institutions (10%):** Vehicle valuation for loans, risk assessment, collateral evaluation.

## 1.3 Why AI is Essential

Used car pricing involves complex, non-linear interactions between features that machine learning models can approximate effectively. SHAP (SHapley Additive exPlanations) provides per-prediction feature contribution analysis, enabling transparent explanations such as: “This car is priced Rs 500,000 higher because: imported assembly (+Rs 300,000), low mileage (+Rs 150,000), automatic transmission (+Rs 50,000).”

# 2. Phase 2: Data Collection & Preprocessing

## 2.1 Data Collection

Data was collected from PakWheels.com using a custom Python scraper (requests + BeautifulSoup4) with rate limiting (2–4 second delays). A two-phase strategy was used: collecting listing URLs from search pages, then extracting detailed specs from individual listings.

| Metric | Value |
|---|---|
| Total pages scraped | 830 |
| Raw records collected | 24,728 |
| Clean records | 24,135 |
| Data retention rate | 97.6% |
| Features per record | 11 base + 15 engineered |
| Time period | 1990–2026 |

*Table 1: Raw Dataset Statistics*

## 2.2 Base Features (11 columns)

| Feature | Type | Description |
|---|---|---|
| make | Categorical | Manufacturer (Toyota, Honda, etc.) |
| model | Categorical | Vehicle model name |
| year | Numeric | Manufacturing year (1990–2026) |
| mileage_km | Numeric | Odometer reading in kilometres |
| engine_cc | Numeric | Engine displacement (cc) |
| assembly | Binary | Local or Imported |
| reg_city | Categorical | Registration city |

| | | |
|---|---|---|
| city | Categorical | Current listing city |
| transmission | Binary | Manual or Automatic |
| fuel_type | Categorical | Petrol/Diesel/Hybrid/CNG |
| price_pkr | Numeric | Target variable (PKR) |

*Table 2: Base Feature Descriptions*

## 2.3 Data Cleaning Pipeline

**1.** Duplicate Removal: 248 exact duplicates removed via drop_duplicates().

**2.** Missing Value Handling: Critical columns (make, model, price) — rows dropped. Numeric — group-wise median imputation. Categorical — group-wise mode imputation.

**3.** Data Type Standardisation: Year, mileage, engine, price cast to int; text stripped and title-cased.

**4.** Outlier Removal: Domain-knowledge bounds (price Rs 100K–50M; year 1990–2026; mileage 0–500K km; engine 100–10,000 cc).

**5.** Text Cleaning: Model names cleaned with regex to remove trailing artefacts.

**6.** Category Standardisation: Assembly, fuel type, and transmission variants normalised.

**7.** Final Validation: Zero missing values confirmed; data type consistency verified.

## 2.4 Feature Engineering (15 additional features)

| Feature | Type | Description |
|---|---|---|
| car_age | Numeric | 2026 – year |
| mileage_per_year | Numeric | Annual average mileage |
| age_group | Categorical | New (≤3 yr) / Mid (4–7 yr) / Old (>7 yr) |
| is_imported | Binary | 1 if imported assembly |
| is_automatic | Binary | 1 if automatic transmission |
| cross_city_sale | Binary | 1 if reg_city ≠ city |
| is_hybrid | Binary | 1 if hybrid fuel type |
| brand_tier | Ordinal | 1=Budget, 2=Premium, 3=Luxury |
| city_tier | Ordinal | 1=Metro, 2=Major, 3=Other |
| engine_category | Categorical | Small/Medium/Large/V.Large |
| price_per_cc | Numeric | Price divided by engine cc |

| | | |
|---|---|---|
| log_price | Numeric | Natural log of price |
| price_category | Categorical | Budget/Mid/Premium/Luxury |
| age_mileage_interact | Numeric | car_age × mileage_km |
| brand_engine_interact | Numeric | brand_tier × engine_cc |

*Table 3: Engineered Features*

### 2.5 EDA Key Findings

| Metric | Value |
|---|---|
| Median price | Rs 3,250,000 |
| Mean price | Rs 4,120,000 |
| Price range | Rs 400,000 – Rs 45,000,000 |
| Mean car age | 6.2 years |
| Top brands | Toyota 38%, Suzuki 24%, Honda 19% |
| Assembly split | Local 64%, Imported 36% |
| Transmission split | Automatic 77%, Manual 23% |

*Table 4: Key EDA Statistics*

Key price correlations: engine_cc (r=0.68), year (r=0.54), car_age (r=–0.52), mileage_km (r=–0.31). Imported cars fetch an 81% premium over local; automatic transmission commands a 45% premium.

## 3. Phase 3: Model Architecture & Implementation

Two gradient boosting models were implemented — XGBoost and LightGBM — with comprehensive hyperparameter tuning via GridSearchCV. Gradient boosting was selected for its proven superiority on tabular data, automatic feature interaction learning, and built-in interpretability.

### 3.1 Hyperparameter Tuning

| Model | Configs Tested | Best Configuration |
|---|---|---|
| XGBoost | 324 | max_depth=7, lr=0.1, n_est=300, subsample=1.0, colsample=0.8 |
| LightGBM | 648 | max_depth=–1, lr=0.05, n_est=300, num_leaves=70, min_child=20, subsample=0.8 |

*Table 5: Hyperparameter Configurations*

### 3.2 Categorical Encoding

Nine categorical features were encoded using sklearn LabelEncoder: make, model, assembly, reg_city, city, transmission, fuel_type, age_group, engine_category. Target-derived features (log_price, price_per_cc, price_category) were excluded to prevent data leakage. A 70/15/15 train/validation/test split was applied with random_state=42.

# 4. Phase 4: Training & Evaluation

| Model | Dataset | MAE (Rs) | RMSE (Rs) | $R^2$ |
|---|---|---|---|---|
| XGBoost | Train | 256,881 | 464,041 | 0.9908 |
| XGBoost | Validation | 515,832 | 1,449,568 | 0.9072 |
| XGBoost | Test | 571,006 | 1,844,519 | 0.8614 |
| LightGBM | Train | 374,429 | 865,449 | 0.9681 |
| LightGBM | Validation | 538,849 | 1,482,601 | 0.9030 |
| LightGBM | Test (Winner) | 562,417 | 1,764,932 | 0.8731 |

*Table 6: Model Performance Results — LightGBM wins (+1.17% $R^2$)*

## 4.1 Feature Importance (LightGBM Top 5)

| Rank | Feature | Importance Score |
|---|---|---|
| 1 | model | 3,769 |
| 2 | engine_cc | 2,734 |
| 3 | mileage_per_year | 1,947 |
| 4 | mileage_km | 1,869 |
| 5 | age_mileage_interact | 1,867 |

*Table 7: LightGBM Feature Importance*

## 4.2 Error Analysis by Price Range (Test Set)

| Price Range | Count | MAE (Rs) | $R^2$ |
|---|---|---|---|
| < Rs 2M | 982 | 287,435 | 0.8523 |
| Rs 2M – Rs 5M | 1,856 | 512,847 | 0.8891 |
| Rs 5M – Rs 10M | 623 | 1,124,562 | 0.8412 |

| | | | |
|---|---|---|---|
| > Rs 10M | 160 | 3,245,891 | 0.7234 |

*Table 8: Error Analysis by Price Range*

Best performance is in the Rs 2M–Rs 5M range (87% of dataset). Performance degrades for the luxury segment (>Rs 10M) due to limited training examples.

# 5. Phase 5: XAI Integration (SHAP)

SHAP (SHapley Additive exPlanations) decomposes each prediction into per-feature contributions grounded in cooperative game theory. For a prediction f(x), SHAP satisfies local accuracy, missingness, and consistency properties.

### *Example Explanation — 2020 Toyota Corolla (Local, 1800cc, 45,000km, Lahore):*

| Factor | SHAP Contribution |
|---|---|
| Base value (mean prediction) | Rs 4,120,000 |
| model (Corolla, premium variant) | +Rs 380,000 |
| engine_cc (1800cc, above median) | +Rs 210,000 |
| mileage_per_year (low annual usage) | +Rs 95,000 |
| car_age (6 years) | –Rs 255,000 |
| Final prediction | Rs 4,550,000 |

*Table 9: SHAP Example Explanation*

# 6. Phase 6: Deployment

The system is deployed as a public Streamlit web application at https://muhammad-ahmad2511-used-car-price-predictor.streamlit.app. The app provides real-time price predictions with SHAP explanations, interactive dropdowns populated with values from the training dataset, and a confidence interval based on model residuals.

| Category | Tools |
|---|---|
| Language | Python 3.11 |
| Web Scraping | BeautifulSoup4, Requests |
| Data Processing | Pandas ≥2.2.0, NumPy ≥2.0.0 |
| Machine Learning | XGBoost ≥2.1.0, LightGBM ≥4.3.0 |
| Explainability | SHAP ≥0.45.0 |

| | |
|---|---|
| Visualisation | Matplotlib ≥3.8.0, Seaborn ≥0.13.0 |
| Frontend / Deployment | Streamlit + Streamlit Cloud |

*Table 10: Technology Stack*

# 7. Application Updates (Post-Submission)

Following initial submission, the Streamlit application (app.py) was iteratively improved to address user-facing issues with input validation, dropdown usability, and encoder compatibility. All changes are backwards-compatible with the trained LightGBM model and label encoders — no retraining was required.

## 7.1 Encoder-Aware Model Dropdowns

The model dropdown now only shows car models that are genuinely known to the trained LabelEncoder. Models that were added speculatively (e.g. BMW X6, MG HS, Haval H6) but were absent from the encoder were removed to prevent silent fallback behaviour and misleading predictions. The Make dropdown no longer includes a generic “Other” option — users must select a real make.

## 7.2 Supported Makes and Known Models

| Make | Known Models in Encoder |
|---|---|
| Toyota | Corolla (all variants), Prius, Aqua, Vitz, Fortuner, Prado, Land Cruiser, Hilux |
| Honda | Civic (all variants), City (all variants), Vezel, Hr-V, Cr-V, Br-V |
| Suzuki | Alto, Wagon R, Cultus, Swift, Mehran, Bolan |
| Kia | Sportage, Picanto, Carnival |
| Hyundai | Tucson (all variants) |
| Daihatsu | Mira, Cuore |
| Mitsubishi | Pajero, Lancer Glx |
| Audi | A3, A4, A6, A8, Q5, Q7, Q8 |
| BMW | 3 Series, 5 Series, 7 Series, X3, X5 |
| Mercedes / Mercedes Benz | C-Class, E-Class, S-Class, GLC, GLE |
| Lexus | Es 350, Lx 570, Rx 350 |
| Volkswagen | Golf, Passat, Tiguan |
| Nissan, MG, Haval, Changan, Chery, Mazda, Subaru, Ford, Jeep, Proton, FAW, DFSK, JAC, BYD, Tesla | Other only |

*Table 11: Supported Makes and Encoder-Known Models*

## 7.3 'Other' Always Last

Both the Make and Model dropdowns now sort entries alphabetically but always pin “Other” to the very bottom, regardless of alphabetical position. This prevents the option from appearing mid-list and improves usability.

## 7.4 Flexible 'Other' Model Mode

When a user selects “Other” as the model — for a make whose specific models are not in the encoder, or for an unlisted variant — the application applies the following rules:

- All fuel types are available: Petrol, Hybrid, Diesel, CNG, Electric, LPG, PHEV.
- Both transmissions are available: Automatic and Manual.
- Engine cc range opens to 660–6000 cc.
- No fuel type or transmission validation is applied at prediction time.
- The encoder maps the input to its known 'Other' class internally — no warnings are shown to the user.

## 7.5 Encoder Fallback (safe_encode)

The safe_encode() helper was updated. Previously it displayed a yellow Streamlit warning banner whenever an unrecognised value was passed to the encoder. The updated version silently falls back to the encoder's 'Other' class if it exists, or index 0 otherwise. No user-visible warning is shown for expected fallbacks.

### *Updated safe_encode() logic:*

```
if val in le.classes_: return encoded value elif 'Other' in le.classes_: return encoded 'Other' else: return 0 # silent fallback
```

## 7.6 Validation Logic

Fuel type and transmission validation at prediction time is now skipped entirely when the selected model is “Other”. For named models, validation still checks the input against the model's known fuel types and transmissions and surfaces a clear error if a mismatch is found, preventing silent bad predictions.

| Model Selection | Fuel/Trans Validation | Engine Range | Fuel Options Shown |
|---|---|---|---|
| Named model (e.g. Corolla) | Strict — must match model's list | Model-specific | Model-specific |
| 'Other' | None — fully open | 660–6000 cc | All 7 types |

*Table 12: Validation Behaviour by Model Selection*

# 8. Team Contributions

| Team Member | Modules & Contributions |
|---|---|
| Muhammad Ahmad 23L-2511 | XGBoost model, hyperparameter tuning, model serialisation (.pkl), Streamlit frontend |
| Muhammad Sufyan 23L-2518 | LightGBM model, hyperparameter tuning, model weights, README, app backend integration |
| Muhammad Daniyal 23L-2600 | Model evaluation, performance comparison, SHAP integration, visualisation, app.py post-submission updates, report writing |

*Table 13: Individual Contributions*

# 9. Conclusion

## 9.1 Summary of Achievements

**Data Pipeline:** Scraped 24,728 records; 97.6% retention after 7-step cleaning; 15 engineered features.

**Model Performance:** LightGBM achieves $R^2$=0.873 and MAE=Rs 562K on the test set.

**Explainability:** SHAP integration provides per-prediction transparency.

**Deployment:** Fully operational Streamlit app on Streamlit Cloud.

**Fairness:** No significant bias detected across city tiers or assembly types.

**App Quality:** Post-submission updates eliminated encoder warnings, fixed validation bugs, and improved dropdown UX.

## 9.2 Limitations

Performance degrades for the luxury segment (>Rs 10M) due to data scarcity.

Model requires periodic retraining as market prices shift.

Scraped data reflects listed (not transacted) prices, which may include negotiation premiums.

Models for makes like MG, Haval, BYD, Tesla not individually represented in the encoder.

## 9.3 Future Work

Neural approaches: TabNet, FT-Transformer for improved luxury-segment accuracy.

Temporal modelling: Incorporate price time-series to capture market trends.

Image features: Integrate vehicle photo analysis for condition assessment.

API endpoint: Expose the model as a REST API for dealer management systems.

Continuous retraining: Automated monthly pipeline with fresh scraped data.

Encoder expansion: Retrain label encoders with new makes/models as data grows.

# References


[1] PakWheels.com — Pakistan's Largest Automotive Marketplace. https://www.pakwheels.com/used-cars/search/-/

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16.

[3] Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.

[4] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 2017.

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.